

QKD Simulation Framework
Master Knowledge Base
Exhaustive Reference for Technical Q&A

Reza Aramjou

August 2026

Contents

1	Theoretical Foundation	2
1.1	Quantum Key Distribution — First Principles	2
1.2	Decoy-State Method — Full Treatment	2
1.2.1	Photon-Number-Splitting Attack & Tagged-Fraction Analysis	3
1.3	Finite-Key Security & Composable Framework	3
1.3.1	Confidence Intervals: Hoeffding, Chernoff, and Clopper-Pearson	4
1.3.2	The Lim 2014 Framework — Deep Dive	4
1.4	The Tight Security Proof	5
1.4.1	Ma (2005) and Wang (2005) Proofs	5
1.5	Multi-Photon Physics	5
2	Methodology & Implementation Architecture	7
2.1	Code Module Hierarchy	7
2.2	Parameter Management & Configuration	7
2.2.1	Detector Types & Imperfections	8
2.2.2	Optical Source Types	8
2.2.3	Channel & Noise Models	9
2.2.4	Polarization Effects	9
2.3	Security Proof Layer — Detailed Architecture	9
2.3.1	Base Class: <code>FiniteKeyProof</code>	9
2.3.2	Proof-Specific Implementations	10
2.4	Linear Programming & SLSQP Optimization	10
2.4.1	Decoy-State LP Solver	10
2.4.2	SLSQP Parameter Optimization	11
2.5	Runner & Orchestration Layer	11
2.5.1	The <code>SimulationRunnerMixin</code> Pattern	11
2.5.2	The Testability Hook: <code>dispatch_work_items</code>	11
2.5.3	Operational Modes	12
2.6	Network Layer — Trusted-Relay QKD Network	12
2.6.1	Node and Link Architecture	12
2.6.2	SPF+ Routing Algorithm	12
2.6.3	KMS (Key Management System)	13
3	Comprehensive Data Analysis & Results	14
3.1	Ten-Campaign Architecture	14
3.2	Key Findings by Campaign	14
3.2.1	C1: Baseline — BB84 + Lim 2014	14
3.2.2	C2: Finite-Key Scaling	14
3.2.3	C3: DWDM Analysis	15
3.2.4	C4: SLSQP Optimization	15
3.2.5	C5: Protocol Comparison	15
3.2.6	C6: Proof Comparison	15

3.2.7	C7: Detector Comparison	15
3.2.8	C8: Confidence Bound Comparison	15
3.2.9	C9: Rogers et al. Validation	15
3.2.10	C10: Sensitivity Analysis	16
3.3	Validation Infrastructure	16
3.3.1	Physics Validation (Z-Test Framework)	16
3.3.2	Cryptographic Validation	16
3.3.3	LP Solver Validation	16
3.3.4	Concurrency Correctness	16
4	Edge Cases & Limitations	17
4.1	Finite-Statistics Regime ($N < 10^5$)	17
4.2	Dark-Count-Dominated Regime ($\eta\mu \ll Y_0$)	17
4.3	Dead-Time Cascades at High Rate	17
4.4	MZM Model Limitations	17
4.5	SLSQP Convergence & Non-Convexity	17
4.6	Trusted-Relay Trust Assumptions	18
4.7	Physical Guardrail Limitation	18
4.8	Afterpulsing Model Limits	18
4.9	Assumption of Phase-Randomized WCP	18
5	The Defense Q&A Bank	19
5.1	Theoretical Foundations & Security Proofs	19
5.2	Implementation & Architecture	20
5.3	Detectors & Physics	21
5.4	Network & Routing	21
5.5	Validation & Testing	22
5.6	Results Interpretation & Future Work	23

Chapter 1

Theoretical Foundation

1.1 Quantum Key Distribution — First Principles

Quantum Key Distribution (QKD) enables two parties, Alice and Bob, to establish a shared secret key whose security rests on the laws of quantum mechanics rather than on unproven computational assumptions. The fundamental principle is the **no-cloning theorem**: an eavesdropper (Eve) cannot create perfect copies of an unknown quantum state. Any measurement she performs necessarily disturbs the state, leaving detectable traces in the statistics observed by the legitimate parties.

The BB84 protocol (Bennett & Brassard, 1984) is the canonical discrete-variable QKD protocol. Alice prepares qubits in one of two mutually unbiased bases — typically the computational basis $Z = \{|0\rangle, |1\rangle\}$ and the Hadamard basis $X = \{|+\rangle, |-\rangle\}$ — and transmits them through a quantum channel. Bob measures each arriving qubit in a randomly chosen basis. After public basis reconciliation (sifting), Alice and Bob retain only those events where their bases matched. They then perform **parameter estimation**: a random subset of the sifted key is sacrificed to estimate the Quantum Bit Error Rate (QBER). If the QBER exceeds a protocol-dependent threshold (approximately 11% for BB84 with one-way post-processing), the protocol aborts. Otherwise, error correction and privacy amplification distill a shorter, perfectly secure key.

The asymptotic secure key rate for BB84 under one-way classical post-processing is:

$$R = q \cdot \{Q_1[1 - H_2(e_1)] - Q_\mu f(E_\mu) H_2(E_\mu)\} \quad (1.1)$$

where q is the basis-sifting factor (typically $1/2$ for BB84), Q_1 is the single-photon gain, e_1 is the single-photon phase-error rate, Q_μ is the total gain, E_μ is the overall QBER, $f(E_\mu) \geq 1$ is the error-correction inefficiency, and $H_2(x) = -x \log_2(x) - (1-x) \log_2(1-x)$ is the binary entropy function.

1.2 Decoy-State Method — Full Treatment

The decoy-state method (Hwang 2003; Lo, Ma & Chen 2005; Wang 2005) addresses the photon-number-splitting (PNS) attack against weak-coherent-pulse (WCP) sources. Alice randomly varies the mean photon number μ of her pulses among several intensity levels: typically a signal intensity μ , a decoy intensity $\nu < \mu$, and the vacuum ($\mu = 0$). Because Eve cannot distinguish which intensity was used for a given pulse, the yields Y_n (probability of a detection conditioned on n photons being sent) and error rates e_n must be identical across intensity settings.

The photon-number distribution for a WCP source follows Poisson statistics:

$$P_\mu(n) = \frac{\mu^n e^{-\mu}}{n!} \quad (1.2)$$

The gain Q_μ (probability that Bob detects a pulse when Alice sends intensity μ) and the overall QBER E_μ are:

$$Q_\mu = \sum_{n=0}^{\infty} Y_n \frac{\mu^n e^{-\mu}}{n!} \quad (1.3)$$

$$E_\mu Q_\mu = \sum_{n=0}^{\infty} e_n Y_n \frac{\mu^n e^{-\mu}}{n!} \quad (1.4)$$

For the standard three-intensity (vacuum + weak decoy + signal) case, closed-form analytic bounds exist:

$$Y_0 = Q_0 = \text{vacuum yield (directly measured)} \quad (1.5)$$

$$Y_1^L \geq \frac{\mu}{\mu\nu - \nu^2} \left(Q_\nu e^\nu - Q_\mu e^\mu \frac{\nu^2}{\mu^2} - \frac{\mu^2 - \nu^2}{\mu^2} Y_0 \right) \quad (1.6)$$

$$e_1^U \leq \frac{E_\nu Q_\nu e^\nu - E_0 Q_0}{Y_1^L \nu} \quad (1.7)$$

where Q_k and E_k denote the gain and QBER for intensity k . The bound in (1.6) is derived from the inequality $Q_\nu e^\nu - Q_\mu e^\mu \frac{\nu^2}{\mu^2} \leq \sum_{n=1}^{\infty} Y_n \frac{\nu^n}{n!} (1 - \frac{\nu^{n-2}}{\mu^{n-2}}) + \frac{\mu^2 - \nu^2}{\mu^2} Y_0$ and keeping only the $n = 1$ term on the RHS yields a valid lower bound.

1.2.1 Photon-Number-Splitting Attack & Tagged-Fraction Analysis

In the PNS attack, Eve blocks single-photon pulses and splits multi-photon pulses, keeping one copy and forwarding the rest to Bob. The fraction of “tagged” (insecure) bits is bounded by the **GLLP (Gottesman-Lo-Lütkenhaus-Preskill)** formula:

$$\Delta = \frac{Q_\mu - Q_1}{Q_\mu} = 1 - \frac{Q_1}{Q_\mu} \quad (1.8)$$

The weak GLLP rate (key rate under the conservative tagged-bit assumption) is:

$$R_{\text{GLLP}} = q \{ -Q_\mu f(E_\mu) H_2(E_\mu) + Q_1 [1 - H_2(e_1)] \} \quad (1.9)$$

This is the origin of Equation (1.1). The term $Q_1 [1 - H_2(e_1)]$ is the usable single-photon entropy; $Q_\mu f(E_\mu) H_2(E_\mu)$ is the leakage from error correction.

1.3 Finite-Key Security & Composable Framework

Practical QKD operates with finite block sizes N , requiring statistical fluctuations to be accounted for. The composable security framework decomposes the total security parameter ε into independent components:

$$\varepsilon = \varepsilon_{\text{sec}} + \varepsilon_{\text{cor}} \quad (1.10)$$

The secrecy error ε_{sec} further decomposes into:

$$\varepsilon_{\text{sec}} = \varepsilon_{\text{PE}} + \varepsilon_{\text{smooth}} + \varepsilon_{\text{PA}} + n_{\text{PE}} \varepsilon \quad (1.11)$$

where ε_{PE} is the parameter-estimation failure probability, $\varepsilon_{\text{smooth}}$ is the smoothing parameter for the min-entropy bound, ε_{PA} is the privacy-amplification failure probability, and $n_{\text{PE}} \varepsilon$ accounts for multiple independent statistical tests.

The finite-key length formula under the composable framework is:

$$\ell = s_{X,0} + s_{X,1}[1 - H_2(\phi_X^U)] - \lambda_{\text{EC}} - 6 \log_2(21/\varepsilon_{\text{sec}}) - \log_2(2/\varepsilon_{\text{cor}}) \quad (1.12)$$

where $s_{X,0}$ and $s_{X,1}$ are the lower bounds on vacuum and single-photon counts in the X basis, ϕ_X^U is the upper bound on the phase-error rate, $\lambda_{\text{EC}} = fE_\mu n_X$ is the error-correction leakage (with $f \approx 1.16$ being the inefficiency factor), and the logarithmic terms account for finite statistics.

1.3.1 Confidence Intervals: Hoeffding, Chernoff, and Clopper-Pearson

Three methods are implemented for computing statistical confidence bounds on observed counts:

Hoeffding Bound: For n independent Bernoulli trials with observed success fraction $\hat{p}$:

$$\Pr[|p - \hat{p}| \geq \delta] \leq 2e^{-2n\delta^2} \quad (1.13)$$

The one-sided confidence interval of width δ at confidence level $1 - \varepsilon$ is $\delta = \sqrt{\frac{\ln(1/\varepsilon)}{2n}}$. This bound is distribution-free but looser than Chernoff for small p .

Chernoff Bound: For Poisson-distributed observations:

$$\Pr[X \leq (1 - \delta)\mathbb{E}[X]] \leq e^{-\delta^2\mathbb{E}[X]/2}, \quad \Pr[X \geq (1 + \delta)\mathbb{E}[X]] \leq e^{-\delta^2\mathbb{E}[X]/3} \quad (1.14)$$

Provides tighter bounds than Hoeffding when the observed mean is small.

Clopper-Pearson (Exact Binomial): Uses the Beta distribution quantiles:

$$p_{\text{lower}} = \text{Beta}^{-1}(\varepsilon/2; k, n - k + 1), \quad p_{\text{upper}} = \text{Beta}^{-1}(1 - \varepsilon/2; k + 1, n - k) \quad (1.15)$$

The most conservative (widest) interval; exact for all sample sizes.

1.3.2 The Lim 2014 Framework — Deep Dive

The Lim et al. (2014) framework (Phys. Rev. A 89, 022307) provides concise, closed-form security bounds for practical decoy-state QKD. Its key innovation is replacing the iterative LP approach with analytic formulas for the standard three-intensity case.

Core Equations (Lim 2014, Eqs. 1–5):

Let $\mu_1 > \mu_2 > \mu_3$ be the three intensities (signal, decoy, vacuum). Define the statistical fluctuation parameters:

$$\tau_n = \frac{\mu_n e^{-\mu_n}}{n!} \quad (1.16)$$

The single-photon yield lower bound:

$$s_{X,1}^L = \frac{\tau_1(\mu_2 - \mu_3)}{\tau_2(\mu_1 - \mu_2)} \left(s_{X,\mu_2}^L e^{\mu_2} - s_{X,\mu_3}^U e^{\mu_3} - \frac{\tau_2(\mu_1 - \mu_3)}{\tau_1(\mu_2 - \mu_3)} Y_0^U \right) \quad (1.17)$$

The single-photon error upper bound:

$$v_{Z,1}^U = \frac{v_{Z,\mu_2}^U e^{\mu_2} - v_{Z,\mu_3}^L e^{\mu_3}}{Y_1^L(\mu_2 - \mu_3)} \quad (1.18)$$

The phase-error rate upper bound:

$$\phi_X^U = \frac{v_{Z,1}^U}{s_{Z,1}^L} + \gamma \left(\varepsilon_{\text{sec}}, \frac{v_{Z,1}^U}{s_{Z,1}^L}, s_{Z,1}^L, s_{X,1}^L \right) \quad (1.19)$$

Hybrid Hoeffding/Chernoff Strategy: The implementation uses an adaptive approach: for intensity settings with large observed counts ($n_{X,\mu_i} > 50$), the Chernoff bound is applied (tighter). For small counts, Hoeffding is used (distribution-free). For zero counts, the vacuum

bound $Y_0^U = -\ln(\varepsilon)/N$ is the analytic limit. This hybrid strategy allows the framework to gracefully handle the entire range from high-count to zero-count regimes.

Kappa-Solver: Instead of fixing ε_{sec} as a user input, the “kappa-solver” iterates over possible values of a security constant κ to maximize the resulting key length while respecting the constraint that total $\varepsilon = 10^{-10}$. This is a one-dimensional search over $\kappa \in [10^{-15}, 10^{-5}]$, evaluating the full key-rate formula at each point. The solver uses bisection with 32 iterations for robust convergence.

Multi-Intensity Extension (> 3 intensities): For four or more intensities, no closed-form analytic bound exists in the Lim 2014 framework. The implementation switches to an LP-based path with non-negativity constraints and a tail-relaxation heuristic. This path is clearly marked in diagnostics as an extension beyond the published paper.

1.4 The Tight Security Proof

The Tight proof uses multiplicative Chernoff bounds rather than additive Hoeffding bounds for statistical estimation of decoy-state parameters. This yields 10–15% higher key rates than Lim 2014, with the advantage growing as the number of pulses N increases.

The multiplicative Chernoff bound for lower-tail estimation of a Poisson mean μ from observed count k :

$$\Pr[X \leq (1 - \delta)\mu] \leq \exp\left(-\frac{\delta^2\mu}{2}\right) \quad (1.20)$$

yielding the one-sided lower confidence bound:

$$\mu^L = \frac{k}{1 + \delta}, \quad \delta = \sqrt{\frac{2 \ln(1/\varepsilon)}{k}} \quad (1.21)$$

The key advantage is that the bound width δ scales as $1/\sqrt{k}$ rather than $1/\sqrt{N}$ (where N is the total number of pulses), making it especially powerful for high-intensity signal pulses with large k .

1.4.1 Ma (2005) and Wang (2005) Proofs

Ma 2005: Uses Gaussian approximations for confidence intervals. The vacuum yield bound:

$$Y_0^U = Q_0 + z_\alpha \sqrt{\frac{Q_0(1 - Q_0)}{N}} \quad (1.22)$$

where z_α is the standard normal quantile. The single-photon bounds use LP with Gaussian-constrained gains.

Wang 2005: A three-intensity decoy-state proof with analytic bounds similar in structure to Lim 2014 but with different coefficient derivations and statistical treatment. Uses a two-decoy variant where both ν_1 and ν_2 are decoy intensities ($\nu_2 < \nu_1 < \mu$). The vacuum yield Y_0 is bounded directly by the weakest decoy.

1.5 Multi-Photon Physics

For a WCP source with mean photon number μ , the probability of emitting exactly n photons follows the Poisson distribution $P(n|\mu) = \mu^n e^{-\mu}/n!$. The multi-photon gain (probability of a detection given n photons were sent) in a channel with transmittance η is:

$$Y_n = 1 - (1 - Y_0)(1 - \eta)^n \quad (1.23)$$

where Y_0 is the dark-count (vacuum) yield. For small η , $Y_n \approx Y_0 + n\eta$.

The overall gain for intensity μ is:

$$Q_\mu = Y_0 + 1 - e^{-\eta\mu}(1 - Y_0) \quad (1.24)$$

and the overall QBER is:

$$E_\mu Q_\mu = e_0 Y_0 + e_{\text{opt}}(1 - e^{-\eta\mu})(1 - Y_0) \quad (1.25)$$

where $e_0 = 1/2$ (random dark-count errors) and e_{opt} is the optical error probability including misalignment: $e_{\text{opt}} = e_{\text{intrinsic}} + (1 - 2e_{\text{intrinsic}}) \sin^2(\theta)$.

Chapter 2

Methodology & Implementation Architecture

2.1 Code Module Hierarchy

The framework is organized into a layered architecture with clear separation of concerns:

Layer	Module	Key Classes/Functions
Orchestration	main_optimized.py	sweep driver, campaign config
Runner	simulation_runner.py	SimulationRunnerMixin, batch dispatch
Dispatch	main_optimized_dispatch.py	dispatch_work_items
Protocol	qkd/protocols.py	BB84, decoy-state logic
Proofs	qkd/proofs/	FiniteKeyProof, Lim2014, Ma2005, Wang2005, Tight
Optimization	qkd/proofs/optimization.py	SLSQP parameter optimizer
LP Engine	qkd/proofs/utls_lp.py	DecoyStateLPAnalyzer
Detectors	qkd/detectors.py	SPD, SNSPD, PNRD models
Sources	qkd/sources.py	WCP, ideal single-photon, heralded
Channel	qkd/channel.py	fiber loss, DWDM, polarization
Data Types	qkd/datatypes.py	all enums, config dataclasses
Parameters	qkd/params.py	QKDParams master config
Network	qkd/network.py	QKDNode, QKDPath, SPF+ routing
Validation	various scripts	Z-test, crypto, LP, concurrency tests

2.2 Parameter Management & Configuration

The master configuration is held in `QKDParams`, a frozen-like dataclass that serves as the single source of truth. The `DEFAULT_CONFIG` dictionary in `main_optimized.py` provides all baseline parameters. Key design principles:

- **Immutability:** Once constructed, parameter objects are effectively immutable. Modifications use `dataclasses.replace()` which returns a new instance, preventing aliasing bugs.
- **Fail-fast validation:** All enums and numeric ranges are validated at construction time via `ParameterValidationError`. Invalid configurations are caught at the API boundary, not deep in simulation logic.
- **Serialization:** All dataclasses implement `to_dict()` / `from_dict()` via the `SerializableDataclassMixin` enabling full round-trip serialization to JSON-compatible dictionaries.

2.2.1 Detector Types & Imperfections

Three detector types are modeled, each as a distinct `DetectorType` enum variant:

Type	Key Parameters	Characteristics
SPD	$\eta \approx 0.15$, $p_{\text{dark}} \approx 10^{-5}$	InGaAs/APD. Room temperature. Low cost.
SNSPD	$\eta \approx 0.50$, $p_{\text{dark}} \approx 10^{-8}$	Superconducting nanowire. Requires cryogenic cooling. Best range.
PNRD	$\eta \approx 0.40$, photon-number resolving	Distinguishes 0,1,2+ photons. Enables advanced protocols.

Detector imperfections modeled in full detail:

- **Dead-time cascades:** After each detection, the detector is blind for a recovery period $\tau_{\text{dead}} \approx 50\text{--}100$ ns. During this window, subsequent photons are missed. The model tracks dead-time state at the nanosecond level and accounts for cascading effects when multiple photons arrive during the recovery window.
- **Afterpulsing:** A detection event has probability $p_{\text{ap}} \approx 0.01\text{--}0.05$ of triggering a spurious secondary detection within the afterpulse window $\tau_{\text{ap}} \approx 200$ ns. The model uses an exponential decay profile.
- **Double-click policy:** When both detectors (0 and 1) click simultaneously, the policy determines how the event is counted. Options: `DISCARD` (ignore), `RANDOM` (assign to random bit), `ALICE_BIASED` (deterministic assignment). The `DISCARD` policy is most conservative and is the default.
- **Electrical noise:** Modeled via `ElectricalNoiseConfig` with additive Gaussian noise on the detector threshold. This affects the effective detection efficiency for near-threshold signals.

2.2.2 Optical Source Types

Four source models are implemented:

- **WCP (Weak Coherent Pulse):** Poisson-distributed photon number with mean μ . Phase-randomized. The workhorse for practical QKD.
- **Ideal Single-Photon:** Exactly one photon per pulse. Used for theoretical benchmarks and upper-bound analysis.
- **Heralded Single-Photon:** Parametric down-conversion (PDC) source where one photon of a pair triggers a “heralding” detector, indicating the presence of its twin. Sub-Poissonian statistics.
- **Phase-Coherent:** Maintains phase coherence across pulses. Used for differential-phase-shift and coherent-one-way protocols.

2.2.3 Channel & Noise Models

Fiber Attenuation: The standard telecom fiber loss model:

$$\eta(L) = 10^{-\alpha L/10} \quad (2.1)$$

with attenuation coefficient $\alpha = 0.2$ dB/km at 1550 nm. This gives $\eta(50 \text{ km}) \approx 0.1$, $\eta(100 \text{ km}) \approx 0.01$.

MZM (Mach-Zehnder Modulator): Modeled via `MZMConfig` with insertion loss, extinction ratio, and finite switching speed. The MZM is the basis-selection device on Bob's side.

DWDM (Dense Wavelength Division Multiplexing): The framework models co-propagation of quantum signals with classical DWDM channels. Inter-channel crosstalk is modeled as an effective noise source:

$$E_{\mu}^{\text{DWDM}} = E_{\mu}^{\text{single}} + E_{\text{crosstalk}} \quad (2.2)$$

where $E_{\text{crosstalk}}$ depends on channel spacing, power levels, and filter isolation. Three power regimes are simulated: -20 dBm (high crosstalk), -34 dBm (moderate), and -40 dBm (low crosstalk, near-isolated).

2.2.4 Polarization Effects

Polarization mode dispersion (PMD) and polarization-dependent loss (PDL) are modeled through Jones matrix calculus. The misalignment angle θ (in radians) couples into the QBER via:

$$E_{\mu} = e_0 + (1 - 2e_0) \sin^2(\theta) \quad (2.3)$$

where e_0 is the intrinsic optical error rate (typically 0.005–0.02).

2.3 Security Proof Layer — Detailed Architecture

2.3.1 Base Class: `FiniteKeyProof`

The abstract base class `FiniteKeyProof` provides the common interface:

- `calculate_key_length(stats_map)` — compute secure key length from tally statistics
- `estimate_yields_and_errors(stats_map)` — estimate Y_1^L , e_1^U from decoy-state analysis
- `explain_key_decision(result)` — human-readable explanation of key/non-key outcome
- `audit_dump(result)` — comprehensive JSON-serializable audit record

The base class enforces: epsilon allocation validation ($\sum \varepsilon_i \leq \varepsilon_{\text{total}}$), numerical safeguards (safe division, log clamping, non-negativity clamping), timing instrumentation, and three operational modes (`PRODUCTION`, `DEBUG`, `AUDIT`).

Key Data Structures:

- **TallyCounts:** frozen dataclass holding `detections` and `errors` for one intensity/basis combination. Enforces `errors ≤ detections`.
- **DecoyEstimates:** holds Y_1^L , e_1^U , feasibility flag, and solver diagnostics.
- **KeyCalculationResult:** final output with `secure_key_length`, `error_correction_leakage`, `privacy_amplification_term`, and `error_codes`.
- **SolverDiagnostics:** LP solver status, iterations, constraint violations, and residual norms.

2.3.2 Proof-Specific Implementations

Lim 2014 (`lim2014.py`):

- Version: `Lim2014Proof` with ~ 800 lines
- Hybrid Hoeffding/Chernoff strategy adaptively selects bound type per intensity
- Kappa-solver: 32-iteration bisection over $\kappa \in [10^{-15}, 10^{-5}]$
- Multi-intensity (> 3) path: LP with non-negativity + tail relaxation
- ε decomposition: $\varepsilon_{\text{sec}}/21$ per single-test, $\ln(21/\varepsilon_{\text{sec}})$ in key formula
- Physical guardrail: $\ell \leq n_X + n_Z$ (key cannot exceed sifted bits)

Tight (`tight.py`):

- Multiplicative Chernoff bounds for all statistical estimates
- 10–15% higher key rate than Lim 2014 at equivalent N
- Advantage scales with N : $\delta \propto 1/\sqrt{k}$ vs $1/\sqrt{N}$

Ma 2005 (`ma2005.py`):

- Gaussian confidence intervals: $Y_0^U = Q_0 + z_\alpha \sqrt{Q_0(1 - Q_0)/N}$
- LP-based decoy-state solver using SciPy `linprog`
- More conservative than Lim 2014 for finite-key scenarios

Wang 2005 (`wang2005.py`):

- Two-decoy variant with analytic bounds
- Vacuum yield bounded by weakest decoy intensity
- Intermediate performance between Ma 2005 and Lim 2014

2.4 Linear Programming & SLSQP Optimization

2.4.1 Decoy-State LP Solver

The `DecoyStateLPAnalyzer` in `utils_lp.py` solves the constrained linear program:

$$\text{Minimize/Maximize } Y_1 \text{ (or } e_1) \quad (2.4)$$

$$\text{Subject to } Q_k^{\text{obs}} - \delta_k \leq \sum_{n=0}^{N_{\text{max}}} Y_n P_k(n) \leq Q_k^{\text{obs}} + \delta_k \quad \forall k \quad (2.5)$$

$$0 \leq Y_n \leq 1, \quad 0 \leq e_n \leq 0.5 \quad \forall n \quad (2.6)$$

where $P_k(n) = \mu_k^n e^{-\mu_k} / n!$ is the Poisson probability for intensity k , δ_k is the statistical fluctuation bound, and N_{max} is the photon-number cutoff (typically 10–15).

2.4.2 SLSQP Parameter Optimization

The parameter optimization in `optimization.py` uses SciPy’s Sequential Least Squares Quadratic Programming (SLSQP) to maximize the secure key rate over a 5-dimensional parameter space:

$$\boldsymbol{\theta}^* = \arg \max_{\boldsymbol{\theta}} R_{\text{secure}}(\boldsymbol{\theta}; L, N, \text{detector_type}) \quad (2.7)$$

where $\boldsymbol{\theta} = [\mu_{\text{signal}}, \mu_{\text{decoy}}, \mu_{\text{vacuum}}, p_{\text{signal}}, p_{\text{decoy}}]$ subject to constraints $\mu_{\text{decoy}} < \mu_{\text{signal}}$, $p_{\text{signal}} + p_{\text{decoy}} + p_{\text{vacuum}} = 1$, and all $\mu_i \geq 0$.

At intermediate distances (40–60 km), SLSQP-optimized intensities yield 15–25% higher key rates compared to heuristically chosen parameters. The optimization landscape is non-convex with multiple local maxima, requiring multiple random restarts (25 starts default) for reliable convergence.

2.5 Runner & Orchestration Layer

2.5.1 The SimulationRunnerMixin Pattern

The runner is implemented as a Mixin class that is composed into the main simulator class. This design choice separates the orchestration logic (parallel dispatch, batch management, seed allocation) from the simulation logic (pulse generation, detection, sifting).

Seed Management Algorithm:

1. A master seed S_0 is provided by the user (or randomly generated).
2. For each batch $i \in [0, B - 1]$, a child seed is derived deterministically:

$$S_i = \text{int}(\text{SHA-256}(S_0 \| i)) \bmod (2^{64} - 1) \quad (2.8)$$

3. Each worker process initializes its own PCG64 RNG with its assigned seed S_i .
4. The deterministic derivation guarantees that re-running with the same S_0 produces identical results regardless of the number of workers or batch size — this is **deterministic reproducibility**.

Batch Processing: The total N pulses are divided into $B = N/b$ batches of size b . Each batch is assigned to a worker process. The optimal batch size $b \approx 10^5$ – 10^6 balances IPC overhead against load-balancing granularity.

ProcessPoolExecutor: Uses Python’s `concurrent.futures.ProcessPoolExecutor` with worker count $w = \min(\text{os.cpu_count}(), 16)$. Each worker runs `_worker_init` which sets `OMP_NUM_THREADS=1` to prevent nested parallelism from oversubscribing cores.

2.5.2 The Testability Hook: `dispatch_work_items`

`main_optimized_dispatch.py` contains the standalone function `dispatch_work_items` that implements the concurrency contract independently of the QKD simulation logic. Key design decisions:

- **Zero QKD dependencies:** The function imports only from the Python standard library, enabling sub-second testing of the dispatch logic with stub workers.
- **Completion contract:** When the function returns, `on_row` has been called for every work item. For multi-worker mode, the `with mp.Pool(...)` context manager guarantees all futures are resolved before exit.
- **Ordering guarantee:** Results are written via `on_row` in submission order (not completion order) using ordered tracking of submitted items.

2.5.3 Operational Modes

The runner supports four operational modes:

- **Single-worker (--workers 1):** All batches run sequentially in the main process. Useful for debugging and small validation runs.
- **Multi-worker (--workers N):** N parallel worker processes. The production mode.
- **Dry-run (--dry-run):** Validates configuration, computes expected batch counts, but performs no simulation. Useful for configuration testing.
- **Resume:** If a previous run was interrupted, the runner detects existing CSV output and resumes from the last completed batch. Requires deterministic seed derivation for correctness.

2.6 Network Layer — Trusted-Relay QKD Network

2.6.1 Node and Link Architecture

The network model uses `QKDNode` objects connected by `QKDLink` edges. Each link maintains:

- A quantum channel (fiber) with distance L_{ij} and attenuation η_{ij}
- A pair of key pools (one per direction) with current capacities K_{ij}^{forward} and K_{ij}^{backward}
- A key generation rate R_{ij} computed by the full physical+proof pipeline

Trusted-Relay Model: Intermediate nodes are assumed honest and physically secure. An end-to-end key between Alice (Node A) and Bob (Node B) is established by:

1. Generating link keys on each hop of the path $A \rightarrow R_1 \rightarrow R_2 \rightarrow \dots \rightarrow R_k \rightarrow B$
2. XOR-forwarding: R_i XORs its two link keys and broadcasts the result
3. Alice and Bob recover the end-to-end key by XOR-combining link keys

The security of this scheme relies on all intermediate nodes being trusted. Compromise of any relay node compromises all keys routed through it.

2.6.2 SPF+ Routing Algorithm

The routing engine extends Dijkstra’s Shortest Path First algorithm with **key-pool watermarks**:

1. Edges where the available key pool $K_{ij} < K_{\text{threshold}}$ are pruned before path computation
2. SPF runs on the pruned graph, yielding the shortest-distance path with sufficient key material
3. If no path exists, the request is queued until key pools recharge

Complexity: $O((V + E) \log V)$ — identical to standard Dijkstra. The pruning step is $O(E)$.

Limitations: SPF+ is not globally optimal. It may route Request A through a path that starves future Request B. True optimality requires multi-commodity flow optimization, which is NP-hard. For metropolitan-scale networks ($V \leq 20$), SPF+ with periodic rebalancing is well within operational requirements.

2.6.3 KMS (Key Management System)

Each node runs a local KMS with:

- Key material organized by (destination, priority) tuples
- Hierarchical key derivation: end-to-end keys $\rightarrow$ session keys $\rightarrow$ traffic keys
- Rate-limited replenishment triggered when key pools drop below watermark
- Audit logging of all key material lifecycle events

Chapter 3

Comprehensive Data Analysis & Results

3.1 Ten-Campaign Architecture

All results are generated by `main_optimized.py` and output as CSV files. Each campaign is a controlled sweep over one or more independent variables. Total: ~ 1800 simulation points.

Table 3.1: Complete Campaign Summary

#	Name	Sweep Variable(s)	Points
1	Baseline BB84+Lim2014	Distance: 0–200 km, $\Delta L = 2$ km	96
2	Finite-Key Scaling	Pulses 10^4 – $10^8 \times 12$ distances	288
3	DWDM Multi-Channel	Power $-20/-34/-40$ dBm $\times 96$ distances	288
4	Auto-Optimized (SLSQP)	Optimized intensities $\times 96$ distances	96
5	Protocol Comparison	4 protocols $\times 96$ distances	384
6	Proof Comparison	4 proofs $\times 96$ distances	384
7	Detector Comparison	SPD/SNSPD/PNRD $\times 96$ distances	288
8	Confidence Bound Comparison	Gaussian/Clopper-Pearson/Hoeffding $\times 96$	288
9	Rogers et al. Validation	Benchmark reproduction $\times 96$ distances	96
10	Sensitivity Grid	QBER $\times$ misalignment 2D sweep	~ 100

3.2 Key Findings by Campaign

3.2.1 C1: Baseline — BB84 + Lim 2014

- Secure key achievable up to ~ 120 km with SPD, ~ 180 km with SNSPD
- Key rate at 50 km: $\sim 10^{-4}$ per pulse (SPD), $\sim 5 \times 10^{-4}$ (SNSPD)
- QBER rises from $\sim 1\%$ at 10 km to $\sim 8\%$ at 100 km (dominated by dark counts at long distances)

3.2.2 C2: Finite-Key Scaling

- At $N = 10^4$, key rate reduced by $\sim 40\%$ vs asymptotic due to statistical fluctuation penalties
- At $N = 10^8$, key rate approaches within 5% of asymptotic limit
- Crossover point (finite-key penalty $< 10\%$): $N \approx 10^6$ for 50 km, $N \approx 5 \times 10^6$ for 100 km

3.2.3 C3: DWDM Analysis

- At -40 dBm per classical channel, QBER increase $< 0.1\%$ — negligible
- At -20 dBm, QBER increase $\sim 2\%$ at 50 km — reduces secure range by ~ 15 km
- Minimum channel spacing for isolation: 100 GHz (0.8 nm) at 1550 nm

3.2.4 C4: SLSQP Optimization

- Optimal $\mu_{\text{signal}} \approx 0.4\text{--}0.6$ (varies with distance)
- Optimal $\mu_{\text{decoy}} \approx 0.05\text{--}0.15$
- 15–25% key-rate improvement over heuristic parameters at 40–60 km
- Improvement diminishes at very short (< 10 km) and very long (> 100 km) distances

3.2.5 C5: Protocol Comparison

- Decoy-state BB84 outperforms non-decoy BB84 by factor $2\text{--}5\times$
- B92 underperforms BB84 (only one basis; lower error tolerance)
- MDI-QKD (stub) shows promise but requires full implementation for comparison

3.2.6 C6: Proof Comparison

- Tight proof: 10–15% higher rate than Lim 2014
- Ma 2005: most conservative (lowest rate, but widest safety margin)
- Wang 2005: intermediate between Ma 2005 and Lim 2014
- Ranking by key rate: Tight $>$ Lim 2014 $>$ Wang 2005 $>$ Ma 2005

3.2.7 C7: Detector Comparison

- SNSPD: $\sim 2\times$ secure range vs SPD (180 km vs 90 km)
- SNSPD advantage from dark-count rate 10^{-8} vs 10^{-5}
- PNRD: marginal range improvement; benefits materialize only for advanced protocols

3.2.8 C8: Confidence Bound Comparison

- Clopper-Pearson: most conservative, widest intervals
- Gaussian: highest key rate for $N \geq 10^6$
- Hoeffding: intermediate; distribution-free guarantee
- Recommendation: CP for $N < 10^5$, Gaussian for $N \geq 10^6$

3.2.9 C9: Rogers et al. Validation

- Key rates reproduce published benchmarks within 3% across all distances
- Confirms correctness of physical models and proof implementations
- Validates the entire pipeline end-to-end against an independent reference

3.2.10 C10: Sensitivity Analysis

- Misalignment $\theta = 0.01$ rad $\rightarrow \sim 50\%$ key-rate reduction (due to $H_2(E)$ steepness near $E = 0$)
- Intrinsic QBER increase from 0.005 to 0.02 $\rightarrow 30\text{--}40\%$ reduction
- Combined worst case ($\theta = 0.02$, QBER = 0.02): system inoperable beyond 30 km

3.3 Validation Infrastructure

3.3.1 Physics Validation (Z-Test Framework)

The `validate_physics.py` script compares simulated channel outputs against analytic predictions using two-sided Z-tests:

$$Z = \frac{\hat{p} - p_0}{\sqrt{p_0(1 - p_0)/N}} \quad (3.1)$$

With significance level $\alpha = 10^{-4}$:

- 96 simulation points per campaign $\times$ multiple campaigns
- Expected false rejections: $96 \times 10^{-4} \approx 0.01$ per campaign
- Tests cover: gain Q_μ vs analytic, QBER E_μ vs analytic, detector efficiency, sifting ratio

3.3.2 Cryptographic Validation

Cross-checks between proof implementations verify internal consistency:

- At the asymptotic limit ($N \rightarrow \infty$), all proofs converge to the same key rate
- Finite-key rates are ordered correctly: Tight $\geq$ Lim2014 $\geq$ Wang2005 $\geq$ Ma2005
- Key-length physical guardrail active: $\ell \leq n_X + n_Z$ for all proofs
- Epsilon budget accounting: $\sum \varepsilon_i \leq 10^{-10}$ verified

3.3.3 LP Solver Validation

`lp_validation.py` tests the decoy-state LP solver:

- Artificial tally statistics generated with known Y_1 , e_1 ground truth
- LP bounds verified to contain the ground truth with probability $\geq 1 - \varepsilon$
- Edge cases: zero decoy counts, single-detection statistics, saturated gains
- All solver status codes verified: optimal, infeasible, unbounded

3.3.4 Concurrency Correctness

`pulse_tracking_trace.py` traces 100 individual pulses through the four-stage pipeline:

1. Generation: photon number, basis, bit value recorded
2. Transmission: channel loss/detection outcome
3. Detection: detector response, dead-time state, afterpulse flag
4. Sifting: basis match, error flag, final tally assignment

End-to-end identity preservation is verified for all 100 traced pulses, confirming no race-condition-induced bias.

Chapter 4

Edge Cases & Limitations

4.1 Finite-Statistics Regime ($N < 10^5$)

At small N , the statistical fluctuation terms dominate the key formula. Specifically, the $\log_2(21/\varepsilon_{\text{sec}})$ term becomes comparable to $s_{X,1}$, potentially driving the key length to zero even when the physical QBER is low. The Clopper-Pearson method is recommended in this regime despite producing lower rates, because its exact binomial coverage is reliable at small N where Gaussian approximations break down.

4.2 Dark-Count-Dominated Regime ($\eta\mu \ll Y_0$)

At long distances where the channel transmittance η is extremely low, the signal-photon detection rate $\eta\mu$ drops below the dark-count rate Y_0 . In this regime:

$$\text{QBER} \rightarrow \frac{e_0 Y_0}{Y_0 + \eta\mu} \approx e_0 = \frac{1}{2} \quad (4.1)$$

No secure key can be generated. The framework correctly reports $\ell = 0$ and identifies `INSUFFICIENT_STATISTICS` as the root cause. SNSPDs extend this threshold by approximately a factor of $\sqrt{Y_0^{\text{SPD}}/Y_0^{\text{SNSPD}}} \approx 30\times$ in distance.

4.3 Dead-Time Cascades at High Rate

At GHz clock rates with high-loss channels, the detector dead time ($\sim 50\text{--}100$ ns) is comparable to the inter-pulse spacing (~ 1 ns). The dead-time cascade model tracks the detector state at each pulse and correctly accounts for the probability that a detection at pulse i blinds the detector for pulses $i+1$ through $i + \lceil \tau_{\text{dead}}/\tau_{\text{pulse}} \rceil$. At 1 GHz, this is up to 100 consecutive pulses per detection. The effective detection rate saturates at $1/\tau_{\text{dead}}$ regardless of incident photon flux.

4.4 MZM Model Limitations

The current MZM model assumes ideal extinction. Real MZMs have finite extinction ratios (20–30 dB), causing basis leakage: a small fraction of photons intended for the Z basis are modulated into the X basis, and vice versa. This leakage is not currently modeled but would manifest as an additional effective QBER contribution of $\sim 10^{-3}$ for a 30 dB extinction ratio.

4.5 SLSQP Convergence & Non-Convexity

The SLSQP optimizer may converge to local maxima in the non-convex 5-parameter landscape. Multiple random restarts (25 default) mitigate but do not eliminate this risk. At certain distance

ranges (particularly 30–40 km), the optimization landscape exhibits narrow ridges where small parameter changes produce large key-rate variations, challenging gradient-based methods.

4.6 Trusted-Relay Trust Assumptions

The trusted-relay network model assumes all intermediate nodes are physically secure and uncompromised. In practice, this requires tamper-proof enclosures, continuous monitoring, and possibly measurement-device-independent (MDI) links between nodes. The framework’s architecture separates the link model from the routing layer, enabling future substitution of quantum repeaters for trusted relays without modifying the routing logic.

4.7 Physical Guardrail Limitation

The physical guardrail $\ell \leq n_X + n_Z$ ensures the key length cannot exceed the number of sifted bits, but it does not account for the possibility that n_X bits may have been consumed by the parameter estimation step. In the current implementation, parameter estimation uses the Z -basis counts while key generation uses X -basis counts; the guardrail is applied post-hoc and may overestimate the available key material if basis allocation is asymmetric.

4.8 Afterpulsing Model Limits

The afterpulsing model uses a single exponential decay with constant probability p_{ap} . Real detectors exhibit multi-exponential afterpulsing with temperature-dependent time constants. The single-exponential model captures first-order effects but may underestimate afterpulse contributions at high count rates ($> 10^7$ cps) where multiple afterpulse cascades overlap.

4.9 Assumption of Phase-Randomized WCP

All decoy-state proofs assume the WCP source is phase-randomized. If the source retains phase coherence across pulses (e.g., gain-switched lasers without external phase modulation), the decoy-state method’s security guarantees may be compromised. The framework does not currently model or detect partial phase correlations between pulses.

Chapter 5

The Defense Q&A Bank

5.1 Theoretical Foundations & Security Proofs

Q1: Why is the no-cloning theorem the foundation of QKD security?

Answer: The no-cloning theorem states that an unknown quantum state cannot be perfectly copied. Since QKD encodes key bits in non-orthogonal quantum states (e.g., $|0\rangle$ and $|+\rangle$), any eavesdropper attempting to intercept and measure these states necessarily introduces disturbances. These disturbances manifest as an increased QBER, which Alice and Bob detect during parameter estimation. Classical cryptography, by contrast, relies on the assumed hardness of mathematical problems (factoring, discrete log) — assumptions that quantum computers invalidate.

Q2: Explain the photon-number-splitting attack and why decoy states defeat it.

Answer: In a PNS attack, Eve exploits the Poisson statistics of a WCP source: some pulses contain multiple photons. Eve can non-destructively measure the photon number (quantum non-demolition measurement), block all single-photon pulses, and split multi-photon pulses, keeping one copy. She then stores her copy and waits for basis revelation during sifting. The decoy-state method defeats this because Eve cannot distinguish signal from decoy pulses. If she blocks single-photon pulses differently based on intensity, the observed yields Q_μ and Q_ν will deviate from the expected Poisson relationship, which Alice and Bob detect. The decoy-state LP bounds Y_1 from below, guaranteeing a minimum single-photon contribution to the secure key.

Q3: What is the advantage of the hybrid Hoeffding/Chernoff strategy in Lim 2014?

Answer: The original Lim 2014 paper uses pure Hoeffding bounds for simplicity — a single δ is applied to all intensities. However, at long distances, decoy and vacuum counts can be very small (sometimes zero). Hoeffding bounds become pathologically loose for small observed counts because the bound width $\delta \propto 1/\sqrt{n}$ overestimates uncertainty when n is near zero. The hybrid strategy applies: (a) Chernoff bounds for moderate-to-large counts ($n > 50$), which are tighter than Hoeffding; (b) Hoeffding for small counts, which remains distribution-free and therefore valid; (c) the analytic vacuum bound $-\ln(\varepsilon)/N$ for zero counts. This adaptive selection provides the tightest available bound at each intensity without compromising security.

Q4: Why does the Tight proof outperform Lim 2014?

Answer: The Tight proof uses multiplicative Chernoff bounds throughout, where the relative uncertainty $\delta \propto 1/\sqrt{k}$ depends on the observed count k , not the total N . For high-intensity signal pulses where k is large, this yields much tighter bounds than additive Hoeffding bounds where $\delta \propto 1/\sqrt{N}$. The 10–15% improvement is most pronounced at intermediate distances (40–80 km).

where signal counts are still substantial, and grows with N . The trade-off is that multiplicative bounds require the observations to be independent Poisson variables — a valid assumption for phase-randomized WCP sources but one that must be verified for other source types.

Q5: What does “composable security” mean and why does it matter?

Answer: Composable security means that the QKD-generated key can be securely used as a component in a larger cryptographic system (e.g., as a one-time-pad key or as input to a symmetric cipher). The security definition is: the QKD output is indistinguishable from an ideal key (uniformly random and independent of Eve’s knowledge) except with probability ε . The composable framework decomposes $\varepsilon = \varepsilon_{\text{sec}} + \varepsilon_{\text{cor}}$ into secrecy and correctness components, and further decomposes ε_{sec} into parameter-estimation, smoothing, and privacy-amplification terms. In the framework, $\varepsilon = 10^{-10}$ is the default total security parameter, and each proof implements a specific decomposition (e.g., Lim 2014 uses $\varepsilon_{\text{sec}}/21$ per statistical test, Tight uses a different allocation).

5.2 Implementation & Architecture

Q6: Why the Mixin pattern for the simulation runner?

Answer: The `SimulationRunnerMixin` separates orchestration concerns from simulation concerns. The simulation class (e.g., `BB84Simulator`) focuses on physics: generating pulses, modeling the channel, detecting, sifting. The Mixin adds orchestration: batch partitioning, seed derivation, parallel dispatch, result aggregation. This separation means: (a) the simulation can be tested independently with single-process runs; (b) the orchestration can be changed (e.g., switching from `ProcessPoolExecutor` to `LokyProcessPoolExecutor` or MPI) without touching the simulation code; (c) the testability hook `dispatch_work_items` can validate the concurrency contract independently with stub workers.

Q7: How does deterministic seed derivation work, and why is it important?

Answer: The master seed S_0 and batch index i are combined via $\text{SHA-256}(S_0||i)$ and reduced modulo $2^{64} - 1$ to produce child seed S_i . This guarantees: (a) every batch gets a unique, non-overlapping seed; (b) re-running with the same S_0 reproduces identical results regardless of batch size, worker count, or operating system; (c) no central seed coordinator is needed — each worker derives its own seed independently. This is critical for scientific reproducibility and for audit scenarios where a third party must be able to reproduce exact simulation outputs.

Q8: What is the “completion contract” and why does it matter?

Answer: The completion contract of `dispatch_work_items` guarantees that when the function returns, `on_row` has been called for every work item. In multi-worker mode, this is enforced by the `with mp.Pool(...)` context manager, which calls `pool.close()` and `pool.join()` on exit, blocking until all submitted futures resolve. Without this contract, a caller might close the CSV file before all rows are written, silently losing data.

Q9: Why is `OMP_NUM_THREADS` set to 1 in worker processes?

Answer: NumPy and SciPy operations can spawn additional threads via OpenMP. If each of w worker processes also spawns t threads, total threads become $w \times t$, potentially exceeding the physical core count and causing severe context-switching overhead. Setting `OMP_NUM_THREADS=1` in each worker prevents this oversubscription. For workloads that benefit from intra-process parallelism (e.g., large LP solves), the value can be tuned with the constraint $w \times t \leq \text{core_count}$.

5.3 Detectors & Physics

Q10: Why does SNSPD double the secure range vs SPD?

Answer: The dark-count rate is the dominant limitation at long distances. SPDs have $p_{\text{dark}} \approx 10^{-5}$ per gate; SNSPDs have $p_{\text{dark}} \approx 10^{-8}$. The effective signal-to-noise ratio at distance L is $S/N \propto \eta(L)\mu/Y_0 = (10^{-\alpha L/10}\mu)/p_{\text{dark}}$. To maintain the same S/N as SPD at distance L , SNSPD can operate at distance L' where $\eta(L') = \eta(L) \times 10^{-3}$, giving $\Delta L = (10/\alpha) \log_{10}(10^3) = 150$ km of additional range. In practice, other noise sources (afterpulsing, electrical noise) reduce the realized gain to approximately 90 km.

Q11: How does the dead-time cascade model work exactly?

Answer: The detector maintains a “blind-until” timestamp. When a photon arrives at time t : (a) if $t < t_{\text{blind}}$, the photon is missed (detector blind); (b) otherwise, a detection is registered with probability η , and if successful, t_{blind} is updated to $t + \tau_{\text{dead}}$. Afterpulsing adds a secondary effect: after each real detection, an afterpulse countdown timer with exponential distribution (mean τ_{ap}) is started; if it fires before the next detection, a spurious count is registered. Both effects are tracked at the individual-pulse level, not through aggregate rate equations.

Q12: What is the double-click policy and which should I use?

Answer: In BB84, Bob uses two detectors (for bit 0 and bit 1). Sometimes both detectors fire from a single pulse (multi-photon pulse, or dark count coinciding with real detection). The **DISCARD** policy ignores these events entirely — most conservative, recommended for security proofs. **RANDOM** assigns a random bit — slightly higher sifting rate but adds $\sim 0.5/p_{\text{multi}}$ to QBER. **ALICE_BIASED** assigns based on Alice’s basis — requires a separate classical channel, not standard.

Q13: How does misalignment affect QBER so dramatically?

Answer: The QBER contribution from misalignment angle θ is $\Delta E = (1 - 2e_0) \sin^2(\theta)$. At small θ , $\sin^2(\theta) \approx \theta^2$, so a misalignment of 0.01 rad ($\approx 0.57^\circ$) produces $\Delta E \approx 10^{-4}$. While this absolute QBER increase seems small, the binary entropy $H_2(E)$ is steepest near $E = 0$: $dH_2/dE = \log_2((1 - E)/E)$. At $E = 0.01$, a 0.0001 increase in E reduces $1 - H_2(E)$ by approximately 0.0014, which for a key rate of 10^{-4} is a 1.4% reduction — small but measurable. The 50% reduction cited in the sensitivity analysis combines misalignment with higher baseline QBER where H_2 is steeper.

5.4 Network & Routing

Q14: Why trusted relays instead of quantum repeaters?

Answer: Quantum repeaters with entanglement swapping and purification are not yet commercially available at the required fidelity and rate. Trusted relays provide a **deployable-now** path to metropolitan QKD networks. The framework’s architecture cleanly separates the link model from the routing layer: **QKDNode** is designed for subtyping; a future **QuantumRepeaterNode** would override only the `simulate_link()` method. The routing layer (SPF+) is link-model-agnostic and would not require modification.

Q15: What are the limitations of SPF+ routing?

Answer: SPF+ extends Dijkstra with key-pool pruning but inherits Dijkstra’s limitations: (a) it is not globally optimal — it greedily routes each request without considering future demand; (b) it does not support multi-path key distribution, which could improve throughput and resilience; (c) pruning based on static thresholds may unnecessarily exclude viable paths when key pools

are near-threshold. Alternatives include: (i) weighted SPF with key-pool-based edge weights (soft pruning), (ii) k -shortest-paths with round-robin, (iii) max-min fair multi-commodity flow (NP-hard but approximable). For metropolitan networks with $V \leq 20$, SPF+ is operationally sufficient.

Q16: How does DWDM affect the security proof?

Answer: The DWDM model adds inter-channel crosstalk as an effective noise source in the QBER: $E_{\mu}^{\text{DWDM}} = E_{\mu}^{\text{single}} + E_{\text{crosstalk}}$. The security proof treats this additional noise as part of the adversary’s attack surface — it reduces the key rate (higher QBER reduces privacy amplification efficiency) but does not require a new proof. The worst-case assumption that Eve controls the crosstalk is conservative. In practice, the classical channels are under the network operator’s control, so this is a defensive assumption.

5.5 Validation & Testing

Q17: How do you guarantee that the simulation is not biased by RNG correlations?

Answer: Each worker receives an independent RNG seeded with a cryptographically derived seed: $\text{seed}_i = \text{SHA-256}(\text{master_seed} || i)$. The SHA-256 hash guarantees that seeds are uniformly distributed and unpredictably related even for consecutive indices. Non-overlapping seed spaces are guaranteed by construction. The concurrency validation module verifies end-to-end identity preservation for 100 traced pulses through the four-stage pipeline, confirming no race-condition-induced bias.

Q18: Why $\alpha = 10^{-4}$ for the physics Z-tests?

Answer: With 96 simulation points per campaign and $\alpha = 10^{-4}$, the expected number of false rejections under the null hypothesis is $96 \times 10^{-4} \approx 0.01$ per campaign — approximately one spurious rejection per 100 campaigns. This is appropriate for scientific validation where false positives (rejecting a correct implementation) waste significant debugging time. The Z-test also reports the actual p -value, enabling manual judgment in borderline cases. A more stringent $\alpha = 10^{-6}$ would nearly eliminate false positives but risk false negatives (accepting an incorrect implementation).

Q19: How does the Rogers et al. validation work?

Answer: Campaign 9 configures the framework to match the parameters, protocol, and proof used in the Rogers et al. (2007) experimental demonstration paper. The simulation outputs (key rate vs distance, QBER vs distance) are compared against the paper’s published figures. Agreement within 3% across all distances confirms that: (a) the physical models (fiber loss, detector efficiency, dark counts) produce correct channel statistics, (b) the Lim 2014 proof implementation correctly computes finite-key rates, and (c) the pipeline has no systematic biases in counting or statistical estimation.

Q20: How do you test the LP solver’s correctness?

Answer: The LP validation uses artificially generated tally statistics with known Y_1 and e_1 ground truth. Tally counts are synthesized by sampling from a Poisson model with the true yields and errors, then the LP solver is run on the sampled counts. The test verifies that the LP bounds contain the true Y_1 and e_1 values with probability $\geq 1 - \varepsilon$. Edge cases include: zero decoy counts (LP should be infeasible), single-intensity statistics (underdetermined system), and saturated gains (all $Y_n = 1$, LP should yield wide but valid bounds).

5.6 Results Interpretation & Future Work

Q21: The 10–15% improvement from Tight seems small. Is it worth the complexity?

Answer: At 80 km where key rates are $\sim 10^{-5}$ per pulse, a 15% gain yields 1.5 additional secure bits per 10^5 pulses. Over 10^9 pulses (seconds at GHz clock rates), this is $\sim 15,000$ additional secure bits — the difference between maintaining or exhausting the key pool. Moreover, the Tight proof’s multiplicative Chernoff bounds become increasingly advantageous as N grows, making it the preferred choice for high-throughput systems.

Q22: SLSQP vs Bayesian optimization — why SLSQP?

Answer: SLSQP is a gradient-based local optimizer that converges quickly (typically < 50 function evaluations) and requires no hyperparameter tuning. Bayesian optimization (e.g., Gaussian Process UCB) would better handle the non-convex landscape, potentially finding better global optima, but requires: (a) a kernel choice (typically Matérn 5/2), (b) an acquisition function (UCB or EI), (c) an exploration-exploitation trade-off, and (d) typically 200–500 function evaluations. SLSQP with 25 random restarts provides a pragmatic balance: most runs find the global optimum, and the computational cost is manageable. A future ML optimization module could add Bayesian methods for users willing to invest the extra computation.

Q23: What is the most impactful next step?

Answer: Three paths, ranked by impact:

(1) **MDI-QKD integration:** Extending the proof layer to measurement-device-independent protocols eliminates all detector-side-channel attacks. Requires modeling a Bell-state measurement at a central untrusted node and adapting the decoy-state bounds. The `mdi.py` stub in `qkd/proofs/` provides a starting point.

(2) **Machine-learning parameter optimization:** Replacing SLSQP with Bayesian optimization for the 5-parameter problem could yield 5–10% additional key-rate improvement, particularly at distance ranges where SLSQP converges to local optima.

(3) **CV-QKD module:** Adding continuous-variable QKD (Gaussian-modulated coherent states) would broaden the framework’s applicability to the other major paradigm, requiring heterodyne detection models and covariance-matrix-based security proofs.

Q24: Why does the framework use CSV output instead of a database?

Answer: CSV provides maximum portability and transparency: the output can be inspected with any text editor, loaded into any analysis tool (Python/pandas, R, MATLAB, Excel), and version-controlled with Git. The complete simulation record is self-contained in a single file. For production deployments, a database (e.g., SQLite for single-node, PostgreSQL for network-scale) could be added as an optional output backend without changing the simulation logic.

Q25: What are the key assumptions that, if violated, break the security proof?

Answer: Five critical assumptions:

1. **Phase-randomized WCP:** The decoy-state method requires that the phase of each pulse is uniformly random and independent. Gain-switched lasers without external phase modulation may retain partial phase correlations between pulses, potentially enabling an unambiguous-state-discrimination attack.

2. **Basis-independent detection efficiency:** Bob’s detectors must have identical efficiency for X and Z basis measurements. Detector efficiency mismatch (“detector efficiency mismatch attack” or “time-shift attack”) can create basis-dependent detection probabilities that Eve can exploit.
3. **Trusted classical channel:** The public discussion during sifting, error correction, and privacy amplification requires an authenticated classical channel. Without authentication, Eve can perform a man-in-the-middle attack, impersonating Bob to Alice and vice versa.
4. **No side channels:** All physical degrees of freedom not explicitly modeled (spectral, temporal, spatial modes) must carry no information about the key bits. Trojan-horse attacks exploit back-reflections into Alice’s source; detector-blinding attacks exploit detector nonlinearities.
5. **Trusted relays are trusted:** In the network model, intermediate nodes are assumed honest and physically secure. Compromise of any relay compromises all keys routed through it. This is an operational assumption, not a cryptographic one.